

# *Section 1: Introduction*

## Evolution of System Design Over the Last 25 Years

---

The system design has evolved dramatically over the last years. What started as simple web applications have transformed into distributed, cloud-native and AI-driven architecture. Let's briefly look at a journey through the history of system design to understand how we have got to a point that we are here. Understanding evolution will help us and appreciate why certain design choices that we make today.

- **1995 - 2005: Early Monolithic Web Applications**

Back in those days, the internet web applications were very simple. Most of the websites followed a monolithic architecture, where everything, frontend, backend and databases are bundled together in a simple application.

- The LAMP stack was the go-to choice. The LAMP stands for Linux, Apache, MySQL and PHP. It was the go-to choice for building applications for most of the organizations.
- Java was predominantly being used by enterprise applications, and .NET was slowly gaining popularity in those days as they started during that time only.
- The system in this era had stateless servers, meaning there were single nodes having databases and web applications and everything.
- Single websites will handle user requests using the server side rendering. There were no real-time communication features.

- **2005 - 2010: Scaling Challenges & Distributed Systems**

This is the era of the emergence of distributed systems. As the internet exploded, businesses like Facebook, Amazon and YouTube faced new challenges.

- Millions of users hitting a system working on a traditional monolithic architecture couldn't keep up with the demand. This led to the rise of distributed systems.
- To improve performance, companies started using caching layers, content delivery networks, and serverless applications to handle more traffic.
- Load balancers became essential to distribute traffic across multiple servers.
- The major shift happened in this era for system design, moving away from single node applications to scalable architecture.

- **2010 - 2015: The Cloud Revolution**

This is the cloud computing revolution.

- Companies like AWS, Azure, Google Cloud Platform have transformed how applications were deployed.
- Instead of managing physical servers, businesses could now scale on demand using cloud services.
- It was this time, we also saw the rise of containerization, which made deployment of applications across different environments just seamless.
- Traditional databases started facing limitations, so it gave rise to NoSQL databases like MongoDB, Cassandra etc.
- Database sharding became a popular technique to handle the massive data sizes.
- The cloud computing helped businesses to now build globally distributed fault-tolerant systems, which laid the foundation of modern system architecture.

- **2015 - 2020: Microservices & Event-Driven Architectures**

As businesses scaled further, monolithic architecture became hard to maintain and deploy. This led to the microservices revolution.

- Instead of one large application, companies started breaking systems into small independent services that could be deployed and scaled separately.
- During this time, we saw the rise of API gateways, service meshes, event driven systems like Kafka, RabbitMQ.
- The shift also brought in the CI/CD Pipelines, which allowed companies to deploy updates faster and more reliably.

- **2020 - Present: Real-Time, AI-driven & Edge Computing**

The era that we are in, the modern era, the era of real-time communications, AI-driven technologies, and edge computing, today the demand on system design is higher than ever. Low latency is required, real-time application communication is required, streaming services are required, AI-driven recommendations are required, instant messaging is required, and many more things that are required right now from the systems.

By default, people expect these features from an application to be there at all times. Technologies like serverless computing, edge computing, Docker containers, Kubernetes, they have made it easier to deploy the applications much closer to the user, reducing this actual latency, increasing the security and observability has gone up in the case of most of the applications, adhering to the modern system design principles.

*Therefore, from monolithic architecture to distributed services to microservices architecture to this world of AI-driven technology, system design has come a long way. Understanding these things can help us understand that system design is a changing landscape. It changed a lot in the last 25 years, and it'll keep on changing. So, it is important to understand the fundamentals and concepts behind system design.*